REPORTE DE PRÁCTICA NO. 3

CONSTRUCCION DE UN AUTOMATA FINITO DETERMINISTA

ALUMNO: Raúl Arana Juárez

Dr. Eduardo Cornejo-Velázquez

1. Introducción

En el área de la teoría de la computación, los autómatas finitos deterministas (AFD) representan uno de los modelos más importantes para el reconocimiento de lenguajes formales. Su aplicación es fundamental en áreas como compiladores, validación de cadenas y diseño de sistemas digitales.
En la presente práctica se desarrolló un programa en lenguaje C++ capaz de simular el comportamiento de un AFD. El sistema permite leer la definición del autómata desde un archivo de entrada, procesar diversas cadenas y determinar si estas pertenecen o no al lenguaje reconocido por dicho autómata.

2. Marco teórico

Un Autómata Finito Determinista (AFD) es un modelo matemático definido por una 5-tupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Donde:

- Σ es el alfabeto de entrada.
- Q es el conjunto finito y no vacío de los estados.
- δ es la función de transición, $\delta : Q \times \Sigma \to Q$.
- $q_0 \in Q$ es el estado inicial.
- $F \subset Q$ es el conjunto de estados finales de aceptación ($F \neq \emptyset$).

Sea $A = (\Sigma, Q, \delta, q_0, F)$ un AFD y sea $w = w_1 w_2 \ldots w_n$ una cadena de símbolos donde $w_i \in \Sigma$.
Entonces, A acepta w si existe una secuencia de estados $r_0, r_1, \ldots, r_n \in Q$ que cumple las siguientes condiciones:

1. $r_0 = q_0$
2. $\delta(r_i, w_{i+1}) = r_{i+1}$ para $i = 0, 1, \ldots, n-1$
3. $r_n \in F$

La condición 1 establece que la máquina comienza en el estado inicial.
La condición 2 establece que la máquina cambia de estado de acuerdo con la función de transición.
La condición 3 establece que la máquina acepta la cadena de entrada si termina en un estado de aceptación.
Por lo tanto, el autómata A reconoce el lenguaje L si:

$$L = \{w \mid Aaceptaw\}$$

El AFD procesa una cadena símbolo por símbolo, cambiando de estado de acuerdo con la función de transición. Si al finalizar la cadena el estado pertenece al conjunto de aceptación, la cadena es aceptada; en caso contrario, se rechaza.

3. Herramientas empleadas

1. Lenguaje C++: Utilizado para implementar el AFD.
2. Compilador (g++): Para compilar el programa.
3. Editor de texto: Para escribir el código y el archivo de entrada.

4. Desarrollo

El programa debe construirse en un lenguaje de programación de alto nivel para implementar un AFD con las siguientes condiciones:
La primera línea contiene los valores (N, S, D, q_0, T, C) que representan los componentes del autómata.
La segunda línea contiene el alfabeto Σ.
La tercera línea contiene los estados de aceptación.
Posteriormente se definen las transiciones del autómata.
Finalmente, se incluyen las cadenas a evaluar.

Implementación en C++

El programa desarrollado sigue el procedimiento anterior utilizando arreglos para representar los componentes del AFD:

- Se utilizó una matriz `trans` para representar la función de transición.
- Un arreglo `finales` para los estados de aceptación.
- Se inicializó la matriz con -1.
- Se procesaron las cadenas carácter por carácter.

El algoritmo sigue los siguientes pasos:

1. Leer los datos del archivo afd.txt
2. Construir la función de transición.
3. Evaluar cada cadena desde el estado inicial.
4. Determinar si el estado final pertenece al conjunto de aceptación.

Código

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream archivo("afd.txt");

    if (!archivo.is_open()) {
        cout << "No se puede abrir el archivo" << endl;
        return 1;
    }

    int N, S, D, q0, T, C;

    archivo >> N >> S >> D >> q0 >> T >> C;

    char alfabeto[100];
    for (int i = 0; i < S; i++) {
        archivo >> alfabeto[i];
    }
```

```
    bool fin[101] = {false};
    for (int i = 0; i < T; i++) {
        int estado;
        archivo >> estado;
        fin[estado] = true;
    }

    int trans[101][256];

    for (int i = 1; i <= N; i++) {
        for (int j = 0; j < 256; j++) {
            trans[i][j] = -1;
        }
    }

    for (int i = 0; i < D; i++) {
        int I, J;
        char X;
        archivo >> I >> X >> J;
        trans[I][(int)X] = J;
    }

    for (int i = 0; i < C; i++) {
        string cadena;
        archivo >> cadena;

        int actual = q0;

        for (int j = 0; j < cadena.length(); j++) {
            char simbolo = cadena[j];

            if (trans[actual][(int)simbolo] != -1) {
                actual = trans[actual][(int)simbolo];
            } else {
                actual = -1;
                break;
            }
        }

        if (actual != -1 && fin[actual]) {
            cout << cadena << " Aceptado" << endl;
        } else {
            cout << cadena << " Rechazado" << endl;
        }
    }

    archivo.close();
    return 0;
}
```

5. Resultados

El programa genero los siguiente resultados:

```
101  ACEPTADA
0    RECHAZADA
11   ACEPTADA
1000100001 ACEPTADA
```

Los resultados coinciden con lo esperado del autómata pero no implementé el generar un archivo de salida por lo que los valores se muestran en la terminal.

6. Conclusiones

La creación del programa permitió comprender de manera práctica los conceptos de estados, transiciones y lenguajes formales. Se concluye que los autómatas finitos deterministas son una herramienta fundamental en ciencias computacionales, con aplicaciones en diversas áreas como análisis léxico y validación de datos.

References

[1] Giró, J., Vázquez, J., Meloni, B., Constable, L. (2015). *Lenguajes formales y teoría de autómatas.* Editorial Alfaomega, Argentina.

[2] Ruiz Catalán, J. (2010). *Compiladores: teoría e implementación.* Editorial Alfaomega, México.

[3] Brookshear, J. G. (1995). *Teoría de la Computación: Lenguajes formales, autómatas y complejidad.* Editorial Addison-Wesley Iberoamericana, U.S.A.